PRÁCTICA 1C - COMPARACIÓN DE ALMACENAMIENTO CON LXC EN PROXMOX

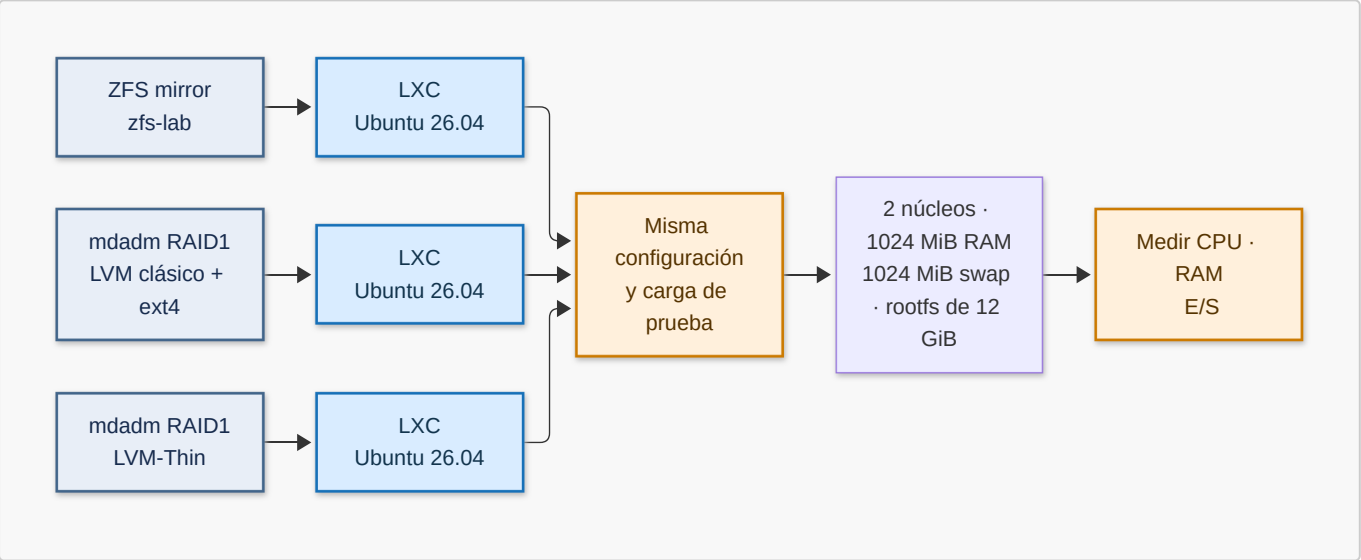
1. Introducción

En esta práctica compararemos el rendimiento y la gestión de contenedores LXC sobre los tres almacenamientos redundantes de la práctica 1b.

Los casos son ZFS mirror (`zfs-lab`), `mdadm` RAID1 + LVM clásico (Directory/ext4, `lvm-raid1-dir`) y `mdadm` RAID1 + LVM-Thin (`lvmthin-raid1`). PaiportArbolado se desplegará en la práctica 1d y no forma parte de esta comparación.

Mediremos CPU, RAM y rendimiento de E/S con un LXC de configuración y carga constantes. No se medirá el consumo eléctrico.

Mapa visual de la práctica 1c



---

2. Objetivos de la práctica

- Crear y configurar un contenedor LXC en Proxmox.
- Comparar rendimiento, consumo de recursos (CPU y RAM) y facilidad de gestión.
- Comparar el rendimiento y la gestión de ZFS, LVM clásico y LVM-Thin para rootfs de LXC.

---

3. Contenedor Linux (LXC)

Un contenedor comparte el kernel del host, pero aísla procesos, redes, almacenamiento y permisos. No emula un hardware completo.

Características principales:

- Comparte el kernel del host y evita virtualizar un sistema operativo completo.
- El consumo depende de los recursos asignados, los servicios activos y la carga; debe medirse.

- Su inicio suele ser rápido.
- Aísla procesos, redes, almacenamiento y permisos.
- Es adecuado para servicios compatibles con el kernel del host y despliegues ligeros.

---

4. Almacenamientos a comparar

Backend de Proxmox	Capas	Cómo almacena el rootfs LXC
ZFS (`zfs-lab`)	ZFS mirror	Dataset/volumen gestionado por el backend ZFS.
Directory (`lvm-raid1-dir`)	`mdadm` RAID1 → LVM clásico → LV → ext4	Archivos dentro del sistema de archivos montado.
LVM-Thin (`lvmthin-raid1`)	`mdadm` RAID1 → PV/VG → thin pool	Volumen thin gestionado como bloque.

---

5. Actividades prácticas a realizar

Configura un contenedor LXC en cada storage y aplica las mismas pruebas y condiciones. **Realiza las mediciones con un único LXC arrancado cada vez**; detén los otros contenedores durante cada medición.

---

6. Medición

6.1 Configuración recomendada

- Usa la misma plantilla Ubuntu Server 26.04.
- Configura 2 núcleos, RAM límite de 1024 MiB, swap de 1024 MiB y rootfs de 12 GiB.
- Selecciona el almacenamiento del caso: `zfs-lab`, `lvm-raid1-dir` o `lvmthin-raid1`. Comprueba que el rootfs del LXC reside en él.

6.2. Qué se va a medir

- **CPU:** carga generada por `sysbench` y uso de CPU del contenedor durante la prueba.
- **RAM:** límite configurado y memoria usada por el contenedor y por el nodo en reposo; el uso del nodo se registra como dato complementario.
- **E/S:** ancho de banda, IOPS y latencias del almacenamiento, medidos con `fio`.
- **Capacidad del almacenamiento:** espacio total, usado y disponible que Proxmox ve en cada backend. En LVM-Thin, además, ocupación de datos y metadatos del thin pool.

6.3. Herramientas de medición

Usa estas mismas herramientas en los tres casos:

Qué medir	Herramienta	Dato
CPU	`sysbench` dentro del CT (apartado 8.1) y vista **CT** →	Eventos por segundo y uso de CPU durante la prueba.

Qué medir	Herramienta	Dato
	Summary en Proxmox	
RAM	Vistas **CT** → **Summary** y **Node** → **Summary** en Proxmox; `free -h` dentro del CT como comprobación opcional	RAM usada por el CT y por el nodo en reposo. El dato del nodo es complementario y puede incluir cachés del host, como la ARC de ZFS.
E/S	`fio` dentro del CT	MiB/s, IOPS y latencia media/p95 de lectura y escritura.
Capacidad	`pvesm status` en la Shell del nodo o **Datacenter** → **Storage**	Estado y espacio total, usado y disponible de cada storage. No mide velocidad.
Ocupación específica	`zpool list` para ZFS; `df -h` para Directory; `lvs` para LVM-Thin	Capacidad del pool, espacio del sistema de archivos o `Data%`/`Meta%` del thin pool. En LVM-Thin, registra esos porcentajes antes de `fio` y después de la prueba, antes de borrar el archivo.

7. Creación del contenedor LXC en Proxmox

Procedimiento

1. En Proxmox, ir a:

```
Datacenter → Node → Create CT
```

2. Elegir la plantilla estándar Ubuntu Server 26.04

3. Configurar:

   - Nombre: usa uno distinto para cada contenedor, por ejemplo `lxc-zfs`, `lxc-lvm-dir` y `lxc-lvmthin`.
   - CTID: asigna a cada contenedor un número que no esté ya en uso en el nodo, por ejemplo `201`, `202` y `203`. Comprueba que esos números estén disponibles antes de crearlos.
   - CPU: 2 núcleos.
   - RAM límite: 1024 MiB y swap: 1024 MiB.
   - Rootfs: 12 GiB, selecciona el almacenamiento del caso (`zfs-lab`, `lvm-raid1-dir` o `lvmthin-raid1`).
   - Red: NIC en `vmbr0` para Internet y NIC en `vmbr1` para el acceso desde el anfitrión; conserva esta configuración en todas las pruebas realizadas.

4. Iniciar el contenedor.

5. Antes de medir, comprueba que el contenedor está iniciado, que tiene la misma configuración en los tres casos y que su rootfs reside en el almacenamiento correspondiente.

Observaciones

LXC comparte el kernel del host. Mantén la configuración del contenedor y cambia únicamente el almacenamiento para que los resultados sean comparables.

---

8. Comparativa de rendimiento

- Instala `fio` en el contenedor y ejecuta las pruebas sobre un archivo del sistema de archivos raíz que resida en el almacenamiento evaluado. No uses `/tmp`.
- Usa los mismos comandos y parámetros en los tres casos.
- Ejecuta cada prueba por separado y espera a que termine.
- No borres el archivo hasta haber registrado la ocupación posterior del almacenamiento; elimínalo al acabar las mediciones del CT.

Ejecuta la instalación dentro de cada CT, no en la Shell del nodo Proxmox:

```
apt update
apt install fio sysbench
```

8.1. Prueba de CPU

Esta prueba compara cuánto trabajo de cálculo puede hacer la CPU en cada CT. `sysbench` repite cálculos de números primos durante un tiempo fijado; no mide la velocidad del almacenamiento ni representa todas las cargas reales. Ejecuta la prueba dentro del CT, no en la shell del nodo Proxmox.

1. Inicia solo el CT que vas a medir y abre su consola.
2. Comprueba que `sysbench` está instalado:

   ```
   sysbench --version
   ```

3. Ejecuta el siguiente comando una vez y espera a que finalice (tarda aproximadamente 30 segundos):

   ```
   sysbench cpu --threads=2 --time=30 run
   ```

El comando configura la prueba así:

- `cpu`: selecciona el benchmark de cálculo de CPU. Por defecto, calcula números primos hasta 20 000 (`--cpu-max-prime=20000`).
- `--threads=2`: usa dos hilos de trabajo simultáneos. No significa que el CT tenga dos núcleos físicos dedicados; Proxmox aplica los límites y la asignación de CPU configurados para ese CT.
- `--time=30`: ejecuta la prueba durante 30 segundos.
- `run`: inicia la ejecución.

Cuando termine, busca en la salida:

- `events per second`: eventos de cálculo completados por segundo. Es el resultado principal para comparar; con la misma configuración, un valor mayor indica más rendimiento de CPU en esta prueba.
- `total number of events`: eventos completados durante la ejecución. Como la duración es fija, debería guardar relación con los eventos por segundo.
- `total time` y `Latency`: duración real y estadísticas de latencia que ofrece `sysbench`. Anótalas si quieres conservar más detalle, pero no sustituyen los eventos por segundo.

Mientras se ejecuta la prueba, observa el uso de CPU en **CT → Summary** y registra el valor o intervalo observado. Es una lectura de Proxmox durante la ejecución, no el resultado `events per second` de `sysbench`. Usa el mismo criterio de observación en cada CT.

Completa una fila por backend:

Backend	Eventos por segundo (`sysbench`)	Eventos totales	Uso de CPU observado en Proxmox
ZFS			
Directory/ext4			
LVM-Thin			

Para que la comparación sea válida, mantén iguales la cantidad/límite de CPU configurado en los CT y el comando utilizado, y evita ejecutar otras pruebas o cargas intensivas durante la medición. Si el resultado varía mucho o hay otra carga en el nodo, espera a que quede libre y repite la medición anotando que la repetiste. El backend de almacenamiento no debería cambiar directamente el resultado de una prueba de cálculo; diferencias grandes pueden deberse a la carga del nodo o a límites de CPU distintos.

8.2. Medición de RAM

- Inicia solo el CT que estés midiendo y espera cinco minutos sin ejecutar pruebas ni cargas adicionales.
- Anota la memoria usada del CT en **CT → Summary** y la RAM usada del nodo en **Node → Summary**. Usa siempre las mismas vistas y unidades en los tres casos.
- `free -h` dentro del CT es una comprobación opcional, no sustituye la medida de **CT → Summary**. No uses `top` o `vmstat` como medida principal: muestran información del sistema o procesos y no son directamente comparables con la memoria total atribuida al CT en Proxmox.
- No midas RAM «durante `sysbench`»: la prueba de CPU no genera una carga de memoria controlada. El dato del nodo incluye el host y puede verse afectado por cachés, incluida la ARC de ZFS; regístralo como dato complementario, no como consumo del CT.

8.4. Pruebas de E/S

Estas pruebas miden la velocidad de lectura y escritura del almacenamiento donde está el rootfs del CT. `fio` trabaja sobre un archivo de prueba, no sobre todo el disco. Ejecuta todos los perfiles dentro del CT y repite exactamente el mismo procedimiento en cada backend.

Antes de empezar

1. Inicia solo el CT que vas a medir y abre una consola dentro de ese CT; no ejecutes estos comandos en la shell del nodo Proxmox.

2. Comprueba que estás en el CT y en el backend correctos. Los comandos usan `/root/fio-testfile`, que ocupa 1 GiB en el rootfs. Asegúrate de que hay espacio libre suficiente.

3. Comprueba que `fio` está instalado:

```
fio --version
```

4. Cierra o detén otras cargas de trabajo que puedan usar el almacenamiento. Ejecuta un único comando de `fio` cada vez y espera a que termine antes de iniciar el siguiente.

Preparar el archivo de prueba

Ejecuta este comando una sola vez al empezar las mediciones de cada CT/backend:

```
fio --name=prepare --filename=/root/fio-testfile --size=1G --rw=write --
bs=1M --ioengine=libaio --direct=1 --end_fsync=1
```

Este paso crea el archivo y escribe sus datos. **No es uno de los resultados que se comparan**: deja preparado el contenido para que la prueba de lectura tenga datos que leer. `--end_fsync=1` solicita que se sincronicen los datos al finalizar. Conserva el archivo durante todas las pruebas y hasta medir la ocupación posterior del almacenamiento.

Ejecutar los perfiles

Copia y ejecuta cada línea por separado. Espera a que `fio` termine y guarda sus resultados antes de pasar a la siguiente:

```
fio --name=seqwrite --filename=/root/fio-testfile --size=1G --rw=write --
bs=1M --iodepth=16 --ioengine=libaio --direct=1 --runtime=30 --time_based
fio --name=seqread --filename=/root/fio-testfile --size=1G --rw=read --
bs=1M --iodepth=16 --ioengine=libaio --direct=1 --runtime=30 --time_based
fio --name=randread4k --filename=/root/fio-testfile --size=1G --rw=randread
--bs=4k --iodepth=16 --ioengine=libaio --direct=1 --runtime=30 --time_based
fio --name=randwrite4k --filename=/root/fio-testfile --size=1G --
rw=randwrite --bs=4k --iodepth=16 --ioengine=libaio --direct=1 --runtime=30
--time_based
```

Qué hace cada perfil:

- `seqwrite`: escribe bloques de 1 MiB de forma secuencial durante 30 segundos.
- `seqread`: lee bloques de 1 MiB de forma secuencial durante 30 segundos.
- `randread4k`: lee bloques de 4 KiB en posiciones aleatorias durante 30 segundos.
- `randwrite4k`: escribe bloques de 4 KiB en posiciones aleatorias durante 30 segundos. Este perfil modifica el archivo de prueba.

Los parámetros comunes indican lo siguiente: `--filename` selecciona el archivo del rootfs que se mide; `--size=1G` limita el área de prueba a 1 GiB; `--runtime=30 --time_based` ejecuta cada perfil durante 30 segundos; `--direct=1` evita medir principalmente la caché de páginas del sistema operativo; `--ioengine=libaio` selecciona el motor de E/S asíncrona y `--iodepth=16` solicita hasta 16 operaciones pendientes. Mantén estos valores iguales para que la comparación sea justa.

Leer y anotar los resultados

Al terminar cada comando, `fio` muestra un resumen. Busca la sección del trabajo con el nombre indicado por `--name`:

- En `seqwrite` y `seqread`, anota `BW` (ancho de banda), en MiB/s. Si `fio` lo muestra en KiB/s, divide el valor entre 1024 para convertirlo a MiB/s. Si la salida también muestra IOPS, puedes anotarlas como dato adicional.
- En `randread4k` y `randwrite4k`, anota `IOPS` y la latencia de finalización (`clat`): `avg` es la media y `95.00th` es el percentil 95 (el 95 % de las operaciones tuvo una latencia igual o menor a ese valor). Conserva la unidad que indique `fio`, normalmente usec (microsegundos).
- No confundas `slat` (tiempo de envío de la operación) con `clat` (tiempo hasta que la operación se completa). Para comparar la latencia solicitada, usa `clat`.

En la tabla general del apartado 11, la latencia se registra en milisegundos (ms). Si `fio` la muestra en microsegundos (usec), divide el valor entre 1000 antes de anotarlo; por ejemplo, `2500 usec = 2,5 ms`. Usa la misma unidad en los tres backends.

Completa la tabla con una fila por backend. Usa la misma unidad y los mismos campos en las tres filas:

Backend	Escritura secuencial BW (MiB/s)	Lectura secuencial BW (MiB/s)	Lectura aleatoria IOPS	Lectura aleatoria clat media / p95	Escritura aleatoria IOPS	Escritura aleatoria clat media / p95
ZFS						
Directory/ext4						
LVM-Thin						

Repite los cuatro perfiles una vez por backend. Si un comando falla, no anotes el error como si fuera un resultado: guarda el mensaje, comprueba que el CT tiene `fio`, espacio libre y el archivo accesible, y repite el perfil cuando hayas corregido la causa. Al acabar y **después de registrar la ocupación posterior** indicada en el apartado 8.5, elimina el archivo dentro del CT:

```
rm /root/fio-testfile
```

8.5. Almacenamiento

Consulta el espacio y el estado de los tres storages con `pvesm status`.

Usa `zpool list` para ZFS, `df -h` para Directory y `lvs` para LVM-Thin. Así obtendrás la capacidad del pool, el espacio del sistema de archivos o `Data%`/`Meta%` del thin pool, según corresponda.

```
lvs -a -o lv_name,vg_name,segtype,lv_size,data_percent,metadata_percent
```

Para más detalle, consulta el espacio del pool ZFS y del sistema de archivos Directory con estos comandos (ajusta la ruta de montaje si es distinta):

```
zpool list zfs-lab
df -h /mnt/raid1-lab
```

`Data%` y `Meta%` solo corresponden al thin pool. En ZFS, la ARC usa RAM del nodo como caché; no atribuyas automáticamente al CT los cambios de memoria del host.

En el thin pool, ejecuta la consulta `lvs` antes de las pruebas de `fio` y repítela después de completar las pruebas, pero antes de borrar `/root/fio-testfile`. Registra ambos valores de `Data%` y `Meta%` en la tabla. Para comparar la capacidad de los storages, anota los valores de `pvesm status` y conviértelos a GiB si aparecen en otra unidad.

---

9. Conclusiones

Compara los tres resultados y responde brevemente:

1. ¿Qué backend obtuvo mejores resultados en CPU, RAM y E/S?
2. ¿Qué diferencias de gestión o capacidad observaste entre ZFS, Directory/ext4 y LVM-Thin?
3. ¿Qué backend elegirías y por qué? Basa la respuesta en tus mediciones; si usaste VirtualBox, limita la conclusión a ese laboratorio.

En LVM-Thin, explica también qué indican `Data%` y `Meta%`. No presupongas que un backend siempre será más rápido: influyen el hardware y la carga.

---

10. Entrega

La entrega de esta práctica es obligatoria para todos los alumnos.

Debe incluir, como mínimo:

- una captura de la configuración del CT para cada backend
- resultados de CPU, RAM y `fio` anotados en la tabla
- capacidad de los storages y, para LVM-Thin, `Data%` y `Meta%`
- respuestas a las preguntas y conclusión basada en los resultados

11. Tabla de resultados

Rellena la tabla para los tres backends. Anota unidades y configuración.

Métrica	LXC - ZFS (`zfs-lab`)	LXC - LVM clásico (`lvm-raid1-dir`)	LXC - LVM-Thin (`lvmthin-raid1`)
CPU/núcleos asignados			
RAM límite (MiB)			
RAM del CT en reposo, vista CT → Summary (MiB)			
RAM del nodo en reposo, vista Node → Summary (MiB)			
Sysbench: eventos/s			
Fio: escritura secuencial (MiB/s)			
Fio: lectura secuencial (MiB/s)			
Fio: lectura aleatoria 4K (IOPS)			
Fio: escritura aleatoria 4K (IOPS)			
Fio: latencia media / p95 de lectura aleatoria (ms)			
Fio: latencia media / p95 de escritura aleatoria (ms)			
Almacenamiento: Total / Used / Avail (GiB)			
Thin pool: Data% / Meta% (antes → después de fio)	N/A	N/A	
Observaciones (ARC, configuración u otros factores)			

`Data%` y `Meta%` solo aplican a LVM-Thin.